Table 7.1 Cedar's core data types with their associated operators and functions

Data type	Operators / Functions	Example
Boolean	`&&, ||, !`	`device.is_managed || resource.category == "public"`
String	`==, !=, like`	`resource.category like "customer-*"`
Long	`==, <, >, <=, >=, +, -, *`	`resource.size <= 1048576 // 1Mb`
Datetime	`==, <, <=, >, >=, datetime()`	`context.requestTime >= datetime("2025-07-01T00:00:00Z")`
Duration	`==, <, <=, >, >=, duration()`	`context.sessionDuration < duration("8h")`
IP address	`==, in, ip()`	`ip(context.clientIp) in ip("10.0.0.0/8")`
Sets	`in, .contains(), .containsAny()`	`principal in Group::"acme-admins"`
Entity types	`is`, type qualifiers	`resource is Document`
Tags	`.hasTag(),.getTag()`	`resource.hasTag("sensitive")`

Each operator in Cedar maps directly to real-world authorization needs. This minimal yet expressive operator set is a key part of what makes Cedar practical and trustworthy for real-world authorization. The next few sections discuss some of Cedar's special operators.

WILDCARD MATCHING

While Cedar doesn't support regular expressions, it does have wildcard matching using the `like` operator. Here's an example in Cedar:

```
context.device.name like "corp-*"
```

This expression returns true whenever the context includes a `device` record whose `name` begins with `"corp-"`, providing reliable wildcard behavior despite the lack of regular expressions.

TIME-BASED CONDITIONS

Time-based constraints are also helpful in many authorization policies for enforcing date or duration limits on access. Here's a policy condition using time-based criteria: